

2520030331 ARNAV CHITIKENA

OS PROGRAM SKILL -3

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_BUFFER_SIZE 64
#define MAX_HISTORY 100

typedef struct HistoryNode {
    char *command;
    struct HistoryNode *next;
    struct HistoryNode *prev;
} HistoryNode;

typedef struct {
    HistoryNode *head;
    HistoryNode *tail;
    HistoryNode *current;
    int count;
} CommandHistory;

void initHistory(CommandHistory *history) {
    history->head = NULL;
    history->tail = NULL;
    history->current = NULL;
    history->count = 0;
}

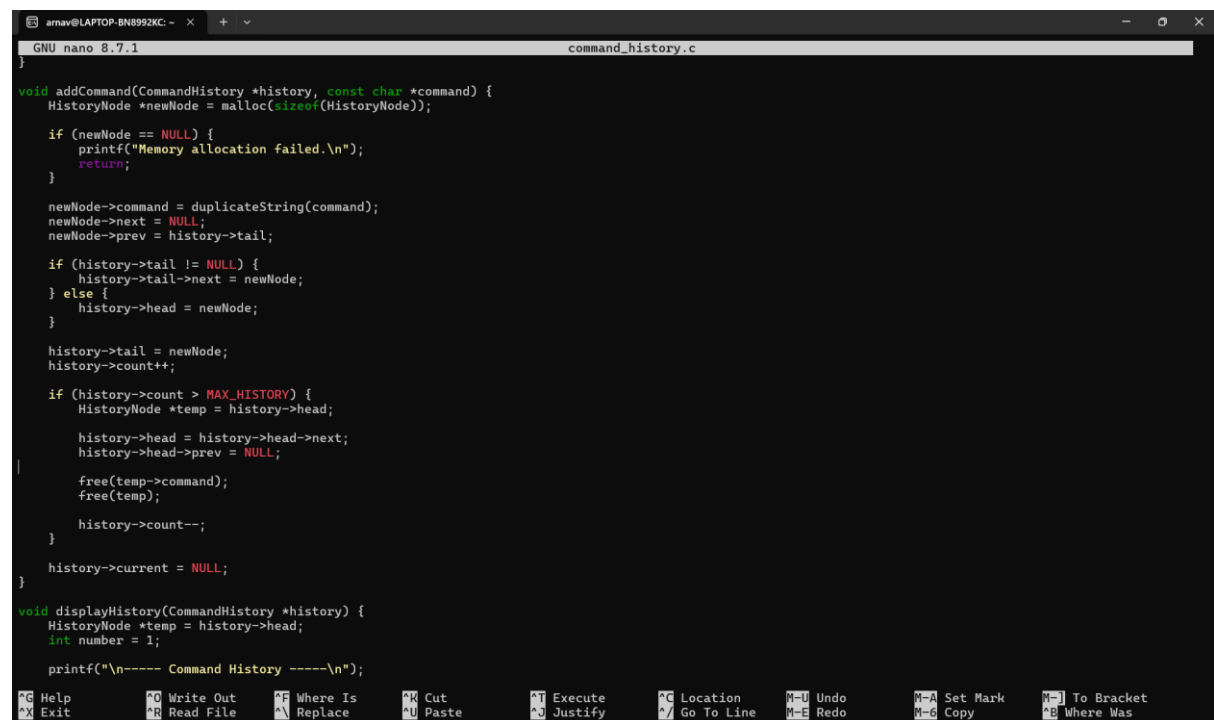
char *duplicateString(const char *str) {
    char *copy = malloc(strlen(str) + 1);

    if (copy == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    strcpy(copy, str);
    return copy;
}

void addCommand(CommandHistory *history, const char *command) {
    HistoryNode *newNode = malloc(sizeof(HistoryNode));

    if (newNode == NULL) {
```



```
    if (newNode == NULL) {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->command = duplicateString(command);
    newNode->next = NULL;
    newNode->prev = history->tail;

    if (history->tail != NULL) {
        history->tail->next = newNode;
    } else {
        history->head = newNode;
    }

    history->tail = newNode;
    history->count++;

    if (history->count > MAX_HISTORY) {
        HistoryNode *temp = history->head;

        history->head = history->head->next;
        history->head->prev = NULL;

        free(temp->command);
        free(temp);

        history->count--;
    }

    history->current = NULL;
}

void displayHistory(CommandHistory *history) {
    HistoryNode *temp = history->head;
    int number = 1;

    printf("\n---- Command History ----\n");
```

amav@LAPTOP-BN8992KC: ~ - x + v

GNU nano 8.7.1 command_history.c

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was

```
arnav@LAPTOP-BN8992KC: ~ - x + v
GNU nano 8.7.1 command_history.c

    while (temp != NULL) {
        printf("%d: %s\n", number, temp->command);
        temp = temp->next;
        number++;
    }

    printf("-----\n");
}

const char *previousCommand(CommandHistory *history) {
    if (history->tail == NULL) {
        return NULL;
    }

    if (history->current == NULL) {
        history->current = history->tail;
    } else if (history->current->prev != NULL) {
        history->current = history->current->prev;
    }

    return history->current->command;
}

const char *nextCommand(CommandHistory *history) {
    if (history->current == NULL) {
        return NULL;
    }

    if (history->current->next != NULL) {
        history->current = history->current->next;
        return history->current->command;
    }

    history->current = NULL;
    return "";
}

void clearHistory(CommandHistory *history) {
    HistoryNode *temp = history->head;

    while (temp != NULL) {
        HistoryNode *next = temp->next;
        free(temp->command);
        free(temp);
        temp = next;
    }

    history->head = NULL;
    history->tail = NULL;
    history->current = NULL;
    history->count = 0;
}

char *readInput(void) {
    size_t size = INITIAL_BUFFER_SIZE;
    size_t length = 0;

    char *buffer = malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (length + 1 >= size) {
            size *= 2;

            char *temp = realloc(buffer, size);

            if (temp == NULL) {
                free(buffer);
                printf("Memory reallocation failed.\n");
                exit(EXIT_FAILURE);
            }

            buffer = temp;
        }

        buffer[length++] = ch;
    }

    buffer[length] = '\0';

    return buffer;
}
```

```
arnav@LAPTOP-BN8992KC: ~ - x + v
GNU nano 8.7.1 command_history.c

    while (temp != NULL) {
        HistoryNode *next = temp->next;

        free(temp->command);
        free(temp);

        temp = next;
    }

    history->head = NULL;
    history->tail = NULL;
    history->current = NULL;
    history->count = 0;
}

char *readInput(void) {
    size_t size = INITIAL_BUFFER_SIZE;
    size_t length = 0;

    char *buffer = malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF) {
        if (length + 1 >= size) {
            size *= 2;

            char *temp = realloc(buffer, size);

            if (temp == NULL) {
                free(buffer);
                printf("Memory reallocation failed.\n");
                exit(EXIT_FAILURE);
            }

            buffer = temp;
        }

        buffer[length++] = ch;
    }

    buffer[length] = '\0';

    return buffer;
}
```

```
arnav@LAPTOP-BN8992KC: ~ - x + v
GNU nano 8.7.1 command_history.c

        buffer = temp;
    }
    buffer[length++] = (char)ch;
}
buffer[length] = '\0';
return buffer;
}

void updateInputBuffer(char **buffer, const char *newCommand) {
    size_t requiredSize = strlen(newCommand) + 1;

    char *temp = realloc(*buffer, requiredSize);

    if (temp == NULL) {
        printf("Unable to update input buffer.\n");
        return;
    }

    *buffer = temp;
    strcpy(*buffer, newCommand);
}

int main(void) {
    CommandHistory history;
    initHistory(&history);

    char *inputBuffer = NULL;

    int choice;

    printf("\n=====n");
    printf("  COMMAND HISTORY MANAGEMENT\n");
    printf("=====n");

    while (1) {
        printf("\n1. Enter Command");
        printf("\n2. Show History");
    }
}

G Help  W Write Out  F Where Is  K Cut  T Execute  C Location  M-U Undo  M-A Set Mark  M-] To Bracket
X Exit  R Read File  A Replace  U Paste  O Justify  / Go To Line  M-E Redo  M-C Copy  B Where Was
```

```
arnav@LAPTOP-BN8992KC: ~ - x + v
GNU nano 8.7.1 command_history.c

printf("\n=====n");
printf("  COMMAND HISTORY MANAGEMENT\n");
printf("=====n");

while (1) {
    printf("\n1. Enter Command");
    printf("\n2. Show History");
    printf("\n3. Previous Command");
    printf("\n4. Next Command");
    printf("\n5. Recall Command");
    printf("\n6. Clear History");
    printf("\n7. Exit");

    printf("\nEnter your choice: ");

    if (scanf("%d", &choice) != 1) {
        while (getchar() != '\n');
        printf("Invalid choice.\n");
        continue;
    }

    getchar();

    switch (choice) {
        case 1:
            printf("Enter command: ");

            inputBuffer = readInput();

            if (strlen(inputBuffer) == 0) {
                printf("Empty command not stored.\n");
                free(inputBuffer);
                inputBuffer = NULL;
                break;
            }

            addCommand(&history, inputBuffer);

            printf("Command stored: %s\n", inputBuffer);
        }
    }
}

G Help  W Write Out  F Where Is  K Cut  T Execute  C Location  M-U Undo  M-A Set Mark  M-] To Bracket
X Exit  R Read File  A Replace  U Paste  O Justify  / Go To Line  M-E Redo  M-C Copy  B Where Was
```

```
GNU nano 8.7.1 command_history.c

    free(inputBuffer);
    inputBuffer = NULL;

    break;

case 2:

    displayHistory(&history);

    break;

case 3: {

    const char *command = previousCommand(&history);

    if (command != NULL) {

        if (inputBuffer == NULL) {
            inputBuffer = malloc(1);

            if (inputBuffer == NULL) {
                printf("Memory allocation failed.\n");
                break;
            }

            inputBuffer[0] = '\0';

            updateInputBuffer(&inputBuffer, command);

            printf("Previous command: %s\n", inputBuffer);
        } else {
            printf("No previous command available.\n");
        }

        break;
    }

case 4: {

    const char *command = nextCommand(&history);

    if (command != NULL) {
```

```
GNU nano 8.7.1 command_history.c

case 4: {

    const char *command = nextCommand(&history);

    if (command != NULL) {

        if (inputBuffer == NULL) {
            inputBuffer = malloc(1);

            if (inputBuffer == NULL) {
                printf("Memory allocation failed.\n");
                break;
            }

            inputBuffer[0] = '\0';

            updateInputBuffer(&inputBuffer, command);

            if (strlen(inputBuffer) == 0)
                printf("Reached end of history.\n");
            else
                printf("Next command: %s\n", inputBuffer);

        } else {
            printf("No next command available.\n");
        }

        break;
    }

case 5: {

    int commandNumber;
    HistoryNode *temp;

    printf("Enter command number to recall: ");

    scanf("%d", &commandNumber);
    getchar();

    if (commandNumber <= 0 ||
```

```
amav@LAPTOP-BN8992KC: ~ - x + v
GNU nano 8.7.1 command_history.c

case 4: {
    const char *command = nextCommand(&history);

    if (command != NULL) {
        if (inputBuffer == NULL) {
            inputBuffer = malloc(1);

            if (inputBuffer == NULL) {
                printf("Memory allocation failed.\n");
                break;
            }

            inputBuffer[0] = '\0';
        }

        updateInputBuffer(&inputBuffer, command);

        if (strlen(inputBuffer) == 0)
            printf("Reached end of history.\n");
        else
            printf("Next command: %s\n", inputBuffer);
    } else {
        printf("No next command available.\n");
    }
    break;
}

case 5: {
    int commandNumber;
    HistoryNode *temp;

    printf("Enter command number to recall: ");
    scanf("%d", &commandNumber);
    getchar();

    if (commandNumber <= 0 ||
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **Undo** **Set Mark** **To Bracket**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line** **Redo** **Copy** **Where Was**

```
amav@LAPTOP-BN8992KC: ~ - x + v
GNU nano 8.7.1 command_history.c

        if (inputBuffer == NULL) {
            printf("Memory allocation failed.\n");
            break;
        }

        inputBuffer[0] = '\0';
    }

    updateInputBuffer(&inputBuffer, temp->command);
    printf("Recalled command: %s\n", inputBuffer);
    break;
}

case 6:
    clearHistory(&history);
    printf("Command history cleared.\n");
    break;

case 7:
    free(inputBuffer);
    clearHistory(&history);

    printf("\nMemory released successfully.\n");
    printf("Program terminated.\n");
    return 0;

default:
    printf("Invalid choice. Try again.\n");
}
}
return 0;
}
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **Undo** **Set Mark** **To Bracket**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line** **Redo** **Copy** **Where Was**

OUTPUT OF SKILL -3

```
arnav@LAPTOP-BN8992KC:~$ cd OS
arnav@LAPTOP-BN8992KC:~/OS$ nano command_history.c
arnav@LAPTOP-BN8992KC:~/OS$ gcc command_history.c -o command_history
./command_history
```

```
=====
COMMAND HISTORY MANAGEMENT
=====
```

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

```
Enter your choice: 1
Enter command: pwd
Command stored: pwd
```

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

```
Enter your choice: 1
```

```
Enter your choice: 1
Enter command: cd documents
Command stored: cd documents
```

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

```
Enter your choice: 2
```

```
----- Command History -----
1: pwd
2: cd documents
-----
```

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

```
Enter your choice: 3
```

----- Command History -----

1: pwd

2: cd documents

1. Enter Command

2. Show History

3. Previous Command

4. Next Command

5. Recall Command

6. Clear History

7. Exit

Enter your choice: 3

Previous command: cd documents

1. Enter Command

2. Show History

3. Previous Command

4. Next Command

5. Recall Command

6. Clear History

7. Exit

Enter your choice: 4

Reached end of history.

4. Next Command
5. Recall Command
6. Clear History
7. Exit

Enter your choice: 4
Reached end of history.

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

Enter your choice: 5
Enter command number to recall: 2
Recalled command: cd documents

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

Enter your choice: 7

```
Enter your choice: 5
Enter command number to recall: 2
Recalled command: cd documents
```

1. Enter Command
2. Show History
3. Previous Command
4. Next Command
5. Recall Command
6. Clear History
7. Exit

```
Enter your choice: 7
```

```
Memory released successfully.
Program terminated.
```

```
arnav@LAPTOP-BN8992KC:~/OS$ |
```